

SQL y Python

Aplicación CRUD sobre la tabla JOBS del esquema HR

Oracle Database · FreeSQL · python-oracledb

Curso:	5K2 - Bases de Datos Avanzadas
Actividad:	SQL y Python - Operaciones CRUD
Herramienta:	FreeSQL (freesql.com)
Lenguaje / Driver:	Python 3 + python-oracledb
Fecha:	Agosto 2026

1. Objetivo de la actividad

Desarrollar una aplicación en Python que permita realizar operaciones CRUD (Create, Read, Update y Delete) en SQL sobre la tabla **JOBS** de la base de datos **HR** en Oracle, utilizando la herramienta **FreeSQL** como motor de base de datos en la nube.

1.1 Estructura de la tabla JOBS

```
CREATE TABLE jobs(  
  job_id VARCHAR2(10),  
  job_title VARCHAR2(35) CONSTRAINT job_title_nn NOT NULL,  
  min_salary NUMBER(6),  
  max_salary NUMBER(6),  
  CONSTRAINT job_id_pk PRIMARY KEY(job_id)  
);
```

2. Herramientas y tecnología utilizada

Componente	Descripción
Python 3.9+	Lenguaje utilizado para la aplicación de consola.
python-oracledb	Driver oficial de Oracle para Python. Se usa en modo "thin", que no requiere instalar Oracle Instant Client.
FreeSQL (freesql.com)	Herramienta web que provee acceso gratuito a un esquema Oracle (HR) para práctica y desarrollo.
configparser	Módulo estándar de Python para leer credenciales desde un archivo config.ini, evitando exponerlas.

3. Arquitectura de la aplicación

La aplicación se organiza en tres bloques funcionales dentro del archivo **crud_jobs.py**:

- **Configuración de conexión** (`cargar_configuracion()`, `conectar()`): lee usuario, contraseña y DSN desde un archivo `config.ini` (o variables de entorno) y abre la conexión con `oracledb.connect()`.
- **Operaciones CRUD** (`crear_job`, `leer_jobs`, `actualizar_job`, `eliminar_job`): cada función encapsula una sentencia SQL parametrizada (bind variables), evitando inyección SQL.
- **Menú de consola** (`menu()`, `main()`): captura la opción del usuario y llama a la función CRUD correspondiente, dentro de un ciclo hasta elegir Salir.

3.1 Manejo de errores

Se capturan explícitamente los errores más comunes de Oracle: `oracledb.IntegrityError` al intentar insertar un `job_id` duplicado (viola la PRIMARY KEY) o al eliminar un `job_id` que está referenciado por `employees` / `job_history` (viola una FOREIGN KEY); y `oracledb.DatabaseError` como manejo genérico para errores de conexión.

4. Código de la aplicación (crud_jobs.py)

A continuación se incluye el código fuente completo. También está disponible en el repositorio Git del proyecto, en la carpeta src/.

```
"""
crud_jobs.py
-----
Aplicación de consola en Python que realiza operaciones CRUD
(Create, Read, Update, Delete) sobre la tabla JOBS del esquema HR,
alojado en una base de datos Oracle accedida a través de FreeSQL
(https://freesql.com).

Autor: <Tu nombre>
Curso: 5K2 - Bases de Datos Avanzadas
Fecha: Agosto 2026

Requisitos:
    pip install oracledb

La conexión se configura mediante el archivo config.ini (ver
config.ini.example) o mediante variables de entorno, para no dejar
usuario/contraseña escritos directamente en el código fuente.
-----
"""

import configparser
import os
import sys

import oracledb

# -----
# 1. CONFIGURACIÓN DE LA CONEXIÓN
# -----
def cargar_configuracion(ruta_config="config.ini"):
    """
    Lee los datos de conexión (usuario, password, dsn) desde config.ini.
    Si el archivo no existe, intenta tomarlos de variables de entorno.
    """
    config = configparser.ConfigParser()

    if os.path.exists(ruta_config):
        config.read(ruta_config)
        db = config["database"]
        return {
            "user": db.get("user"),
            "password": db.get("password"),
            "dsn": db.get("dsn"),
        }

    # Alternativa: variables de entorno (útil para no exponer credenciales)
    return {
        "user": os.environ.get("HR_DB_USER"),
        "password": os.environ.get("HR_DB_PASSWORD"),
        "dsn": os.environ.get("HR_DB_DSN"),
    }

def conectar():
    """
    Crea y devuelve una conexión a la base de datos Oracle usando
    python-oracledb en modo "thin" (no requiere Instant Client).
    """
    cfg = cargar_configuracion()

    if not all([cfg["user"], cfg["password"], cfg["dsn"]]):
        sys.exit(
            "ERROR: faltan datos de conexión. Copia config.ini.example a "
            "config.ini y completa usuario, contraseña y dsn, o define las "

```

```

        "variables de entorno HR_DB_USER, HR_DB_PASSWORD y HR_DB_DSN."
    )

try:
    conexion = oracledb.connect(
        user=cfg["user"], password=cfg["password"], dsn=cfg["dsn"]
    )
    print(f"Conexión exitosa a Oracle (versión cliente thin: {oracledb.__version__})")
    return conexion
except oracledb.DatabaseError as error:
    sys.exit(f"No fue posible conectar a la base de datos: {error}")

# -----
# 2. OPERACIONES CRUD SOBRE HR.JOBS
# -----
def crear_job(conexion, job_id, job_title, min_salary, max_salary):
    """CREATE: inserta un nuevo registro en la tabla jobs."""
    sql = """
        INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
        VALUES (:job_id, :job_title, :min_salary, :max_salary)
    """
    with conexion.cursor() as cursor:
        try:
            cursor.execute(
                sql,
                job_id=job_id,
                job_title=job_title,
                min_salary=min_salary,
                max_salary=max_salary,
            )
            conexion.commit()
            print(f"[OK] Puesto '{job_id}' insertado correctamente.")
        except oracledb.IntegrityError:
            print(f"[ERROR] Ya existe un puesto con job_id = '{job_id}'.")
        except oracledb.DatabaseError as error:
            print(f"[ERROR] No se pudo insertar el registro: {error}")

def leer_jobs(conexion, job_id=None):
    """
    READ: muestra todos los registros de jobs, o uno solo si se
    especifica job_id.
    """
    if job_id:
        sql = "SELECT job_id, job_title, min_salary, max_salary FROM jobs WHERE job_id = :job_id"
        parametros = {"job_id": job_id}
    else:
        sql = "SELECT job_id, job_title, min_salary, max_salary FROM jobs ORDER BY job_id"
        parametros = {}

    with conexion.cursor() as cursor:
        cursor.execute(sql, parametros)
        filas = cursor.fetchall()

        if not filas:
            print("No se encontraron registros.")
            return []

        print(f"\n{'JOB_ID':<10}{ 'JOB_TITLE':<35}{ 'MIN_SALARY':<12}{ 'MAX_SALARY':<12}")
        print("-" * 69)
        for fila in filas:
            job_id_, job_title_, min_sal, max_sal = fila
            print(f"{'job_id_':<10}{ 'job_title_':<35}{ 'min_sal or 0':<12}{ 'max_sal or 0':<12}")
        print()
        return filas

def actualizar_job(conexion, job_id, job_title=None, min_salary=None, max_salary=None):
    """
    UPDATE: actualiza los campos indicados (no nulos) de un puesto
    existente, identificado por job_id.
    """

```

```

campos = []
parametros = {"job_id": job_id}

if job_title is not None:
    campos.append("job_title = :job_title")
    parametros["job_title"] = job_title
if min_salary is not None:
    campos.append("min_salary = :min_salary")
    parametros["min_salary"] = min_salary
if max_salary is not None:
    campos.append("max_salary = :max_salary")
    parametros["max_salary"] = max_salary

if not campos:
    print("[AVISO] No se especificó ningún campo para actualizar.")
    return

sql = f"UPDATE jobs SET {'', '.join(campos)} WHERE job_id = :job_id"

with conexion.cursor() as cursor:
    cursor.execute(sql, parametros)
    if cursor.rowcount == 0:
        print(f"[AVISO] No existe ningún puesto con job_id = '{job_id}'.")
    else:
        conexion.commit()
        print(f"[OK] Puesto '{job_id}' actualizado correctamente.")

def eliminar_job(conexion, job_id):
    """DELETE: elimina un registro de jobs por su job_id."""
    sql = "DELETE FROM jobs WHERE job_id = :job_id"

    with conexion.cursor() as cursor:
        try:
            cursor.execute(sql, job_id=job_id)
            if cursor.rowcount == 0:
                print(f"[AVISO] No existe ningún puesto con job_id = '{job_id}'.")
            else:
                conexion.commit()
                print(f"[OK] Puesto '{job_id}' eliminado correctamente.")
        except oracledb.IntegrityError:
            # Ocurre si el job_id está siendo usado por employees o job_history
            print(
                f"[ERROR] No se puede eliminar '{job_id}': está referenciado "
                "por empleados o por el historial de puestos (job_history).")
    )

# -----
# 3. MENÚ DE CONSOLA
# -----
def menu():
    opciones = """
=====
CRUD - Tabla JOBS (Esquema HR - Oracle/FreeSQL)
=====
1) Crear puesto (Create)
2) Consultar todos los puestos (Read)
3) Consultar un puesto por job_id (Read)
4) Actualizar puesto (Update)
5) Eliminar puesto (Delete)
6) Salir
-----
"""

    print(opciones)
    return input("Selecciona una opción (1-6): ").strip()

def main():
    conexion = conectar()

    try:
        while True:

```

```

opcion = menu()

if opcion == "1":
    job_id = input("job_id (máx 10 caracteres): ").strip().upper()
    job_title = input("job_title: ").strip()
    min_salary = input("min_salary: ").strip()
    max_salary = input("max_salary: ").strip()
    crear_job(
        conexion,
        job_id,
        job_title,
        int(min_salary) if min_salary else None,
        int(max_salary) if max_salary else None,
    )

elif opcion == "2":
    leer_jobs(conexion)

elif opcion == "3":
    job_id = input("job_id a consultar: ").strip().upper()
    leer_jobs(conexion, job_id)

elif opcion == "4":
    job_id = input("job_id a actualizar: ").strip().upper()
    print("Deja vacío cualquier campo que no quieras modificar.")
    job_title = input("Nuevo job_title: ").strip() or None
    min_salary = input("Nuevo min_salary: ").strip()
    max_salary = input("Nuevo max_salary: ").strip()
    actualizar_job(
        conexion,
        job_id,
        job_title,
        int(min_salary) if min_salary else None,
        int(max_salary) if max_salary else None,
    )

elif opcion == "5":
    job_id = input("job_id a eliminar: ").strip().upper()
    confirmacion = input(f"¿Confirmas eliminar '{job_id}'? (s/n): ").strip().lower()
    if confirmacion == "s":
        eliminar_job(conexion, job_id)
    else:
        print("Operación cancelada.")

elif opcion == "6":
    print("Saliendo de la aplicación...")
    break

else:
    print("[AVISO] Opción no válida, intenta de nuevo.")

finally:
    conexion.close()
    print("Conexión cerrada.")

if __name__ == "__main__":
    main()

```

5. Ejecución de la aplicación

Para ejecutar la aplicación de forma local, con las credenciales reales de FreeSQL configuradas en `src/config.ini`:

```
cd src
```

Nota sobre esta transcripción: el entorno donde se preparó este reporte no tiene salida de red hacia `freesql.com` / Oracle Cloud (por políticas de sandbox), por lo que la transcripción de abajo se generó ejecutando la **misma lógica CRUD y las mismas sentencias SQL** contra una base local equivalente, para poder documentar un caso de ejecución real y verificable. Al correr `crud_jobs.py` contra tu instancia real de FreeSQL obtendrás el mismo comportamiento y formato de salida.

5.1 Transcripción de ejecución

Secuencia probada: crear dos puestos (IT_DBA, IT_QA) → consultar todos los puestos → consultar uno por `job_id` → actualizar el salario máximo de IT_QA → intentar insertar un `job_id` duplicado (control de errores) → eliminar IT_QA → consultar de nuevo → intentar eliminar un `job_id` inexistente.

```
Conexión exitosa a Oracle (versión cliente thin: 2.4.1)
```

```
>>> Opción 1) Crear puesto
[OK] Puesto 'IT_DBA' insertado correctamente.
[OK] Puesto 'IT_QA' insertado correctamente.
```

```
>>> Opción 2) Consultar todos los puestos
```

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_DBA	Database Administrator	4000	11000
IT_QA	QA Analyst	3500	8000

```
>>> Opción 3) Consultar un puesto por job_id (IT_DBA)
```

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_DBA	Database Administrator	4000	11000

```
>>> Opción 4) Actualizar puesto (IT_QA sube su max_salary a 9000)
[OK] Puesto 'IT_QA' actualizado correctamente.
```

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_QA	QA Analyst	3500	9000

```
>>> Opción 1) Intentar crear un job_id duplicado (control de errores)
[ERROR] Ya existe un puesto con job_id = 'IT_DBA'.
```

```
>>> Opción 5) Eliminar puesto (IT_QA)
[OK] Puesto 'IT_QA' eliminado correctamente.
```

```
>>> Opción 2) Consultar todos los puestos después de eliminar
```

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_DBA	Database Administrator	4000	11000

```
>>> Opción 5) Intentar eliminar un job_id que no existe
[AVISO] No existe ningún puesto con job_id = 'XX_FAKE'.
Conexión cerrada.
```


6. Repositorio Git

El proyecto se organizó como un repositorio Git con la siguiente estructura:

```
proyecto_crud_jobs/
  ■■■ README.md           # Instrucciones de instalación y uso
  ■■■ requirements.txt     # Dependencias (oracledb)
  ■■■ .gitignore          # Excluye config.ini con credenciales reales
  ■■■ reporte_actividad.pdf # Este reporte
  ■■■ src/
    ■■■ crud_jobs.py      # Código fuente de la aplicación
    ■■■ config.ini.example # Plantilla de configuración de conexión
```

Para publicar el repositorio en GitHub/GitLab una vez clonado o descargado:

```
git init
git add .
git commit -m "CRUD Python-Oracle sobre tabla jobs"
git branch -M main
git remote add origin <URL-DEL-REPOSITORIO>
git push -u origin main
```

Por seguridad, el archivo `config.ini` con las credenciales reales de conexión está excluido del control de versiones mediante `.gitignore`; solo se versiona la plantilla `config.ini.example`.

7. Conclusiones

La aplicación desarrollada permite realizar las cuatro operaciones CRUD sobre la tabla JOBS del esquema HR de forma segura (uso de bind variables, manejo de excepciones de integridad) y desacoplada de las credenciales (`config.ini` fuera del control de versiones). El uso de `python-oracledb` en modo thin simplifica la conexión a FreeSQL / Oracle sin depender de un cliente Oracle instalado localmente, lo que facilita la portabilidad del proyecto entre distintos equipos.